

Microservices Architecture

Backend decomposition — Auth & Onboarding, Recommendation, Main/Core, Payment

Version 1.0 · Draft · Companion to the PRD and Database Design v2.0

1. Decomposition overview

The backend splits into four services, not three. Auth & Onboarding, Recommendation, and Main/Core form one natural boundary around the product experience; Payment is deliberately isolated as its own service because it carries a different compliance perimeter (PCI-adjacent obligations), a different failure mode (money must never silently double-process), and a different traffic shape (bursty webhook delivery vs. steady booking traffic). Each service owns its own database — no cross-service foreign keys. Anything one service needs to know about another's data arrives as an event or an API call, never a join.

1.1 Recommended rollout

- Deploy Payment as its own service and its own database from day one — this is the boundary that matters most and is cheapest to get right early.
- Auth & Onboarding, Recommendation, and Main/Core can ship as one deployable unit with three internal modules and three separate schemas at first. Splitting them into independent deployments later is a redeploy, not a redesign, provided the schema and API boundaries below are respected from day one.

2. Service boundaries & data ownership

Service	Owns	Database
Auth & Onboarding	accounts, learner_profiles, assessments, assessment_subjects, teachers, teacher_subjects, teacher_curricula, teacher_availability, subjects, curricula	PostgreSQL
Recommendation	matches, plus local read models: learner_profile_view, teacher_profile_view (synced via events, never queried live from Auth)	PostgreSQL (+ pgvector/OpenSearch if matching becomes similarity-based)
Main / Core	sessions, feedback, progress_reports, ratings	PostgreSQL
Payment	payments, payouts, payment_methods, payment_providers	PostgreSQL — isolated instance/cluster, minimal IAM surface, append-only ledger discipline

Recommendation and Main never query Auth's tables directly. Auth publishes `LearnerProfileUpdated` and `TeacherVerified` events; each downstream service keeps only the denormalized fields it actually needs to do its job.

3. Inter-service communication

Interaction	Pattern	Why
Client → any service	Sync REST/gRPC via API gateway	Normal request/response; the user is waiting.
Auth → Recommendation	Async event (<code>LearnerProfileUpdated</code> , <code>TeacherVerified</code>)	Not time-critical; decouples Recommendation's availability from Auth's.
Recommendation → Main (match accepted)	Sync call to create a pending session	Simplest correctness story for the first hop of the saga; see section 5.
Main → Payment (session booked)	Sync call to initiate payment	User is on a payment screen waiting for a result.
Payment → Main (charge outcome)	Async event (<code>PaymentSucceeded</code> , <code>PaymentFailed</code>)	Payment gateways are themselves async (webhooks); Main reacts, doesn't poll.

Interaction	Pattern	Why
Main → Payment (session completed)	Async event (SessionCompleted)	Drives payout calculation; not on the user's critical path.

4. Booking saga & compensation

No transaction can span three databases, so booking a session is a choreographed saga:

- 1. Recommendation marks the match accepted.
- 2. Main creates a sessions row with status = 'pending' — booked, not yet confirmed.
- 3. Payment attempts the charge against the provider and emits PaymentSucceeded or PaymentFailed.
- 4. Main reacts: success → sessions.status = 'confirmed'; failure → sessions.status = 'cancelled' (the compensating action — there is nothing to roll back because nothing was ever finalized).

4.1 Idempotency

Payment gateways retry webhooks. The unique index on (provider_id, provider_reference) — see section 7 — is what stops a retried webhook from charging or confirming a session twice. Every cross-service call in this saga should carry an idempotency key, not just the payment step.

4.2 The outbox pattern

If a service writes a status change and then crashes before publishing the event that should follow it, you get a stuck record — a session marked pending forever with no charge ever attempted. The fix: write the event to a local outbox table in the same transaction as the status change; a separate relay process publishes from the outbox to the broker. Each service that publishes events owns its own outbox table.

```
CREATE TABLE outbox_events (
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  aggregate_type VARCHAR(50) NOT NULL,      -- e.g. 'session', 'payment'
  aggregate_id BIGINT NOT NULL,
  event_type   VARCHAR(100) NOT NULL,      -- e.g. 'PaymentSucceeded'
  payload      JSONB NOT NULL,
  created_at   TIMESTAMPTZ NOT NULL DEFAULT now(),
  published_at TIMESTAMPTZ,
  CONSTRAINT chk_published_after_created
    CHECK (published_at IS NULL OR published_at >= created_at)
);
CREATE INDEX ix_outbox_unpublished ON outbox_events (created_at)
WHERE published_at IS NULL;      -- relay process polls this
```

5. Event catalog

Event	Producer	Consumer(s)	Key payload fields	Delivery
LearnerProfileUpdated	Auth	Recommendation	profile_id, grade, subjects, curriculum_id, budget_tier	≥ once
TeacherVerified	Auth	Recommendation	teacher_id, verification_status, subjects, curricula	≥ once
MatchAccepted	Recommendation	Main	match_id, learner_profile_id, teacher_id	sync call
PaymentSucceeded	Payment	Main	payment_id, session_id, provider_reference, amount_minor	≥ once
PaymentFailed	Payment	Main	payment_id, session_id, provider_reference, failure_reason	≥ once
SessionCompleted	Main	Payment	session_id, teacher_id, scheduled_at	≥ once

“≥ once” (at-least-once) delivery is assumed throughout — every consumer must be written to safely process the same event twice.

6. API contracts (representative)

6.1 Auth & Onboarding

```
POST /accounts
POST /accounts/login
POST /learner-profiles
POST /learner-profiles/{id}/assessments
GET /teachers/{id}/verification-status
POST /teachers/{id}/verify (staff-only)
```

6.2 Recommendation

```
POST /matches (trigger matching for a learner_profile_id)
GET /matches?learner_profile_id=
POST /matches/{id}/accept
POST /matches/{id}/decline
```

6.3 Main / Core

```
POST /sessions (from an accepted match)
GET /sessions/{id}
PATCH /sessions/{id}/status
POST /sessions/{id}/feedback
GET /learner-profiles/{id}/progress
```

6.4 Payment

```
POST /payments (initiate charge for a session or plan)
GET /payments/{id}
POST /webhooks/{provider} (inbound: paystack, flutterwave, stripe)
POST /payouts (staff/scheduled)
GET /teachers/{id}/earnings
```

7. Payment provider integration

This is the part of the Payment service that touches the outside world, and the part most likely to cause a silent financial bug if under-specified.

7.1 Recommended providers

Market	Provider(s)	Notes
Nigeria (launch)	Paystack (primary), Flutterwave (fallback)	Both cover card, bank transfer, USSD, and mobile money — the full method list from the guided assessment / payment step.
Ghana, Kenya, South Africa	Flutterwave	Broadest single-provider coverage across these markets; avoids a third integration at expansion time.
UK, Canada, US (diaspora)	Stripe	Neither Paystack nor Flutterwave has full first-party coverage in these markets.

7.2 The provider_reference field

payments.provider_reference stores the payment processor's own transaction identifier — Paystack's reference, Flutterwave's tx_ref/flw_ref, Stripe's payment_intent id. This is the reconciliation key: when finance or support investigates a dispute, they match this value against the provider's own dashboard, not the internal payments.id, which the provider has never seen.

One refinement over the v2 schema: separate how a parent paid (card, bank transfer, USSD, mobile money — `payment_methods`) from who processed it (Paystack, Flutterwave, Stripe — a new `payment_providers` table). The same method can flow through different providers, and `provider_reference` should be unique per provider, not globally, in case two providers ever coincidentally issue an overlapping reference string.

```
CREATE TABLE payment_providers (  
  id          BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,  
  code        VARCHAR(30) NOT NULL UNIQUE,      -- 'paystack' | 'flutterwave' | 'stripe'  
  display_name VARCHAR(60) NOT NULL,  
  webhook_secret_ref VARCHAR(120) NOT NULL,      -- pointer into a secrets vault, never the raw secret  
  is_active   BOOLEAN NOT NULL DEFAULT true  
);  
  
ALTER TABLE payments  
  ADD COLUMN provider_id BIGINT NOT NULL REFERENCES payment_providers(id) ON DELETE RESTRICT;  
  
ALTER TABLE payments DROP CONSTRAINT IF EXISTS ux_payments_provider_reference;  
ALTER TABLE payments  
  ADD CONSTRAINT ux_payments_provider_ref_per_provider  
  UNIQUE (provider_id, provider_reference);      -- scoped per provider, not global
```

7.3 Webhook handling flow

- 1. Client initiates payment → Payment service calls the provider's initialize/charge API → receives a `provider_reference` immediately → writes a payments row with status = 'processing'.
- 2. The provider processes the charge and calls back to `POST /webhooks/{provider}`.
- 3. Payment service verifies the webhook signature (provider-specific HMAC secret, retrieved via `webhook_secret_ref`) before trusting the payload — an unverified webhook is not evidence of anything.
- 4. Payment service looks up the payments row by (`provider_id`, `provider_reference`), updates status, and writes an outbox event: `PaymentSucceeded` or `PaymentFailed`.
- 5. If the same `provider_reference` arrives twice (a provider retry, which will happen), the unique index plus an idempotent update (only transition from 'processing', never re-apply to an already-'success' row) prevents double-processing.

7.4 Reconciliation

Webhooks can be missed — network blips, a deploy mid-flight. A nightly reconciliation job calls each active provider's list-transactions API and diffs the result against the local payments table by (`provider_id`, `provider_reference`), flagging any row where the provider shows a final state but the local row is still 'processing'. This is the safety net underneath the webhook flow, not a replacement for it.

7.5 Provider abstraction

The Payment service should define a thin adapter interface — `initialize()`, `verify()`, `refund()` — implemented once per provider, rather than branching on provider code throughout the business logic. This keeps Paystack, Flutterwave, and Stripe pluggable: adding a provider later, or running two concurrently for reliability (Paystack primary, Flutterwave automatic fallback on repeated initialize failures), becomes a new adapter, not a rewrite.

8. Operational notes

- Each service migrates its own schema independently, with its own versioned migration history — no shared migration run touches two services' tables.
- Distributed tracing (OpenTelemetry) is not optional once a booking spans four services — a stack trace from one service no longer tells the whole story.
- The Payment service's network segment should be the narrowest of the four: it needs to reach the payment providers and the message broker, and nothing else needs to reach it except through the API gateway and the broker.
- Route dashboard/reporting reads to a replica per service, as already noted in the database design — this holds independently once each service has its own database.

9. Open decisions

- Recommendation → Main for MatchAccepted is modeled as synchronous above for simplicity. If matching volume grows large enough that Main's availability shouldn't gate Recommendation's, this can move to an event with Main creating the pending session asynchronously — revisit once real traffic patterns exist, not preemptively.
- Whether Paystack or Flutterwave is primary for the Nigerian launch is a commercial decision (settlement speed, fee structure, support responsiveness) as much as a technical one — the adapter pattern in 7.5 means this choice isn't a lock-in either way.
- A dedicated reference-data service for subjects/curricula was considered and rejected at this scale — Auth & Onboarding owns them and publishes changes; revisit only if a fifth domain genuinely needs them independently of Auth.